4 Class files and bytecode

This chapter covers

- Class loading
- Reflection
- The anatomy of class files
- JVM bytecode and why it matters

One tried-and-true way to become a more well-grounded Java developer is to improve your understanding of how the platform works. Getting familiar with core features such as class loading and the nature of JVM bytecode can greatly help with this goal.

Consider the following scenarios that a senior Java developer might encounter: Imagine you have an application that makes heavy use of dependency injection (DI) techniques such as Spring, and it develops problems starting up and fails with a cryptic error message. If the problem is more than a simple configuration error, you may need to understand how the DI framework is implemented to track down the problem. This means understanding class loading.

Or suppose that a vendor you're dealing with goes out of business. You're left with a final drop of compiled code, no source code, and patchy documentation. How can you explore the compiled code and see what it contains?

All but the simplest applications can fail with a `ClassNotFoundException` or `NoClassDefFoundError`, but many developers don't know what these are, what the difference is between them, or even why they occur.

This chapter focuses on the aspects of the platform that underlie these concerns. We'll also discuss some more advanced features, but they are intended for those folks who like to dive deep and can be skipped if you're in a hurry.

We'll get started with an overview of class loading—the process by which the JVM locates and activates a new type for use in a running program. Central to that discussion

are the `Class` objects that represent types in the JVM. Next, we'll look at how these concepts build into the major language feature known as reflection (or Core Reflection).

After that, we'll discuss tools for examining and dissecting class files. We'll use `javap`, which ships with the JDK, as our reference tool. Following this class file anatomy lesson, we'll turn to bytecode. We'll cover the major families of JVM opcodes and look at how the runtime operates at a low level.

Let's get started by discussing *class loading*—the process by which new classes are incorporated into a running JVM process. In this section, we will first discuss the basics of "classic" class loading, as it was done in Java 8 and earlier. Later in the chapter, we will talk about how the arrival of the modular JVM introduces some (small) changes to class loading.

4.1 Class loading and class objects

A .class file defines a type for the JVM, complete with fields, methods, inheritance information, annotations, and other metadata. The class file format is well-described by the standards, and any language that wants to run on the JVM must adhere to it.

NOTE A class is the fundamental unit of program code that the Java platform will understand, accept, and execute.

From the perspective of a beginning Java developer, a lot of the class loading mechanism is hidden from view. The developer provides either an executable JAR file or the name of the main application class (which must be present on the classpath), and the JVM finds and executes the class.

Any application dependencies (e.g., libraries other than the JDK) must also be on the classpath, and the JVM finds and loads them as well. However, the Java specifications do not say whether this needs to be done at application startup or later, as needed.

NOTE The API that the Java class loading system presents to the user is fairly simple—a lot of the complexity is hidden on purpose, and we will discuss the developer-available API later in the chapter.

Let's start with a very simple example:

```
Class<?> clazz = Class.forName("MyClass");
```

This piece of code will load a class, `MyClass`, into the current execution state. From the JVM's perspective, to achieve this a number of steps must be performed. First, a class file corresponding to the name `MyClass` must be found, and then the class it contains it must be resolved. These steps are performed in native code—in HotSpot, the native method is called `JVM_DefineClass()`.

The actual process, at a high level, is that the native code builds the JVM's internal representation (which is called a *klass* and which is not a Java object—we will meet it properly in chapter 17). Then, provided the klass can be extracted from the class file successfully, the JVM constructs a Java "mirror" of the klass, which is passed back to Java code as a `Class` object.

After this, the `Class` object representing the type is available to the running system, and new instances of it can be created. In the previous example, `clazz` ends up holding the `Class` object corresponding to the type `MyClass`. It cannot hold the klass, because a klass is a JVM-internal object and *not* a Java object.

NOTE The same process is used for the main application class, all of its dependencies, and any other classes that may be required after the program has started.

In this section, we'll cover the steps from the JVM's point of view in a bit more detail and provide an introduction to class loaders, which are the objects that control this entire process.

1.1 Loading and linking

One way of looking at the JVM is that it is an execution container. In this view, the purpose of the JVM is to consume class files and execute the bytecode they contain. To achieve this, the JVM must retrieve the contents of the class file as a data stream of bytes, convert it to a useable form, and add it to the running state. This is essentially the process that we are describing here.

This somewhat complex process can be divided in a number of ways, but we refer to it as *loading* and *linking*.

NOTE Our discussion of loading and linking refers to some details that are specific to the HotSpot code, but other implementations should do similar things.

The first step is to acquire the data stream of bytes that constitute the class file. This process starts with a byte array that is often read in from a filesystem (but other alternatives are definitely possible).

Once we have the stream, it must be parsed to check that it contains a valid class file structure (this is sometimes called *format checking*). If so, then a candidate klass is created. During this phase, while the candidate klass is being filled in, some basic checks are performed (e.g., can the class being loaded actually access its declared superclass? Does it try to override any `final` methods?).

However, at the end of the loading process, the data structure corresponding to the class isn't usable by other code yet, and, in particular, we don't have a fully functional klass.

To get there, the class must now be linked and then initialized before it can be used. Logically speaking, this step breaks down into three subphases: verification, preparation, and resolution. However, in a real implementation, the code might not be cleanly separated out, so if you are planning to read the source code, you should be aware that the description provided here is a high-level or conceptual description of the process and does not have a precise correlation to the actual implementing code.

With this in mind, verification can be understood to be the phase that confirms that the class conforms to the requirements of the Java specifications and won't cause runtime errors or other problems for the running system. This relationship between the phases of linking can be seen in figure 4.1.

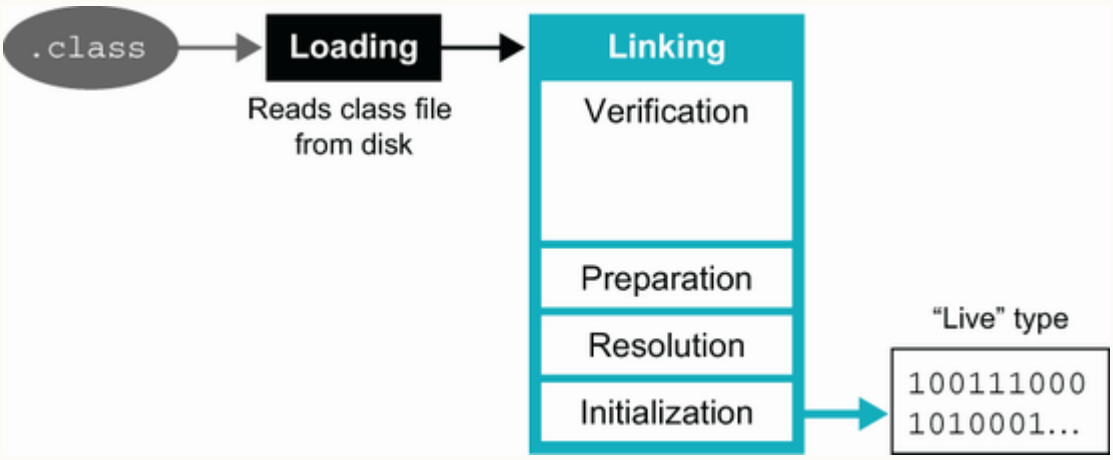


Figure 4.1 Loading and linking (with subphases of linking)

Let's meet each phase in turn.

Verification

Verification is quite a complex process, consisting of several independent concerns. For example, the JVM needs to check that symbolic information contained in the constant

pool (discussed in detail in section 4.3.3) is self-consistent and obeys the basic behavior rules for constants.

Another major concern, and probably the most complex part of verification, is checking the bytecode of methods. This involves ensuring that the bytecode is well-behaved and doesn't try to circumvent the JVM's environmental controls.

Some of the main checks that are performed follow:

- Make sure bytecode doesn't try to manipulate the stack in disallowed or evil ways.
- Make sure every branch instruction (e.g., from an `if` or a loop) has a proper destination instruction.
- Check methods are called with the right number of parameters of the correct static types.
- Check local variables are assigned only suitably typed values.
- Check each exception that can be thrown has a legal catch handler.

These checks are done for several reasons, including performance. The checks enable the skipping of runtime checks, thus making the interpreted code run faster. Some of them can also simplify the compilation of bytecode into machine code at runtime (*just-in-time compilation*, which we'll cover in chapter 6).

Preparation

Preparing the class involves allocating memory and getting static variables in the class ready to be initialized, but it doesn't initialize variables or execute any JVM bytecode.

Resolution

Resolution is the part of linking where the JVM checks that the supertype of the class being linked (and any interfaces that it implements) are already linked, and if they are not, then they are linked before the linking of this class continues. This can lead to a recursive linking process for any new types that have not been seen before.

NOTE A key phrase that relates to this aspect of class loading is the *transitive closure* of types. Not only the types that a class inherits from directly but also all types that are indirectly referenced must be linked.

Once all additional types that need to be loaded have been located and resolved, the JVM can initialize the class it was originally asked to load.

In this final phase, any static variables are initialized and any static initialization blocks are run. This is a significant point because it is only now that the JVM is finally running bytecode from the newly loaded class.

When this step completes, the class is fully loaded and ready to go. The class is available to the runtime and new instances of it can be created. Any further class loading operations that refer to this class will now see that it is loaded and available.

1.2 Class objects

The end result of the linking and loading process is a `Class` object, which represents the newly loaded and linked type. It's now fully functional in the JVM, although for performance reasons, some aspects of the `Class` object are initialized only on demand.

NOTE `Class` objects are regular Java objects. They live in the Java heap, just like any other object.

Your code can now go ahead and use the new type and create new instances. In addition, the `Class` object of a type provides a number of useful methods, such as `getSuperclass()`, which returns the `Class` object corresponding to the supertype.

Class objects can be used with the Reflection API for indirect access to methods, fields, constructors, and so forth. A `Class` object has references to `Method`, `Field`, and various other objects that correspond to the members of the class. These objects can be used in the Reflection API to provide indirect access to the capabilities of the class, as we will see later in this chapter. You can see the high-level structure of this in figure 4.2.

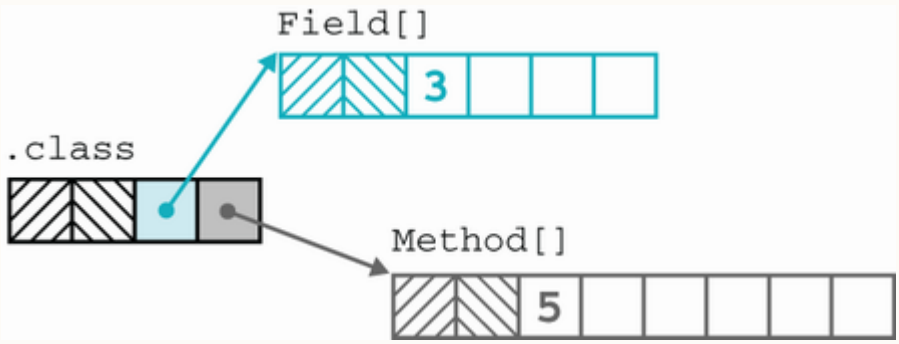


Figure 4.2 `Class` object and `Method` references

So far, we haven't discussed exactly which part of the runtime is responsible for locating and linking the byte stream that will become the newly loaded class. This is handled by

class loaders—subclasses of the abstract class `ClassLoader`, and they're our next subject.

4.2 Class loaders

Java is a fundamentally object-oriented system with a dynamic runtime. One aspect of this is that Java's types are alive at runtime, and the type system of a running Java platform can be modified—in particular, by the addition of new types. The types that make up a Java program are open to extension by unknown types at runtime (unless they are `final` or one of the new `sealed` classes). The class-loading capability is exposed to the user. Class loaders are just Java classes that extend `ClassLoader`—they are themselves Java types.

NOTE In modern Java environments, all class loaders are modular. Loading classes is always done within the context of a module.

The class `ClassLoader` has some native methods, including the loading and linking aspects that are responsible for low-level parsing of the class file, but user class loaders are not able to override this aspect of class loading. It is not possible to write a class loader using native code.

The platform ships with the following typical class loaders, which are used to do different jobs during the startup and normal operation of the platform:

- `BootstrapClassLoader` (or *primordial* class loader)—This is instantiated very early in the process of starting up the JVM, so it's usually best to think of it as being a part of the JVM itself. It's typically used to get the absolute basic system loaded—essentially `java.base`.
- `PlatformClassLoader`—After the bare minimum system has been bootstrapped, then the platform class loader loads the rest of the platform modules that the application depends upon. This class loader is the primary interface to access any platform class, regardless of whether it was actually loaded by this loader or the bootstrap. It is an instance of an internal class.
- `AppClassLoader`—The application class loader—this is the most widely used class loader. It loads the application classes and does the majority of the work in most modern Java environments. In a modular JVM, the application class loader is no longer an instance of `URLClassLoader` (as it was in Java 8 and earlier) but, instead is an instance of an internal class.

Let's see these new class loaders in action, by adding some code to the top of the `main` method in `SiteCheck` from the `wgjd.sitecheck` module from chapter 2:

```
...
var clThis = SiteCheck.class.getClassLoader();
System.out.println(clThis);
var clObj = Object.class.getClassLoader();
System.out.println(clObj);
var clHttp = HttpClient.class.getClassLoader();
System.out.println(clHttp);
....
```

We recompile it with the following:

```
$ javac -d out wgjd.sitecheck/module-info.java \
      wgjd.sitecheck/wgjd/sitecheck/*.java \
      wgjd.sitecheck/wgjd/sitecheck/*/*.java
```

and run it like this:

```
$ java -cp out wgjd.sitecheck.SiteCheck http://github.com/well-grounded-java
```

Notice the use of the "starting module" syntax rather than an explicit starting class.

This produces the next output:

```
jdk.internal.loader.ClassLoaders$AppClassLoader@27705Odc
null
jdk.internal.loader.ClassLoaders$PlatformClassLoader@12bb4df8
http://github.com/well-grounded-java: HTTP_1_1
```

The class loader for `Object` (which is in `java.base`) reports as `null`. This is a security feature—the bootstrap class loader does no verification and provides full security access to every class it loads. For that reason it does not make sense to have the class loader represented and available within the Java runtime—too much potential for bugs or abuse.

In addition to their core role, class loaders are also often used to load resources (files that aren't classes, such as images or config files) from JAR files or other locations on

the classpath. This is often seen in a pattern that combines with try-with-resources to produce code like this:

```
try (var is = TestMain.class.getResourceAsStream("/resource.csv");
    var br = new BufferedReader(new InputStreamReader(is));) {
    // ...
}
// Exception handling elided
```

The class loaders provide this mechanism in a couple of different forms, returning either a `URL` or an `InputStream`.

2.1 Custom class loading

More complex environments will often have a number of additional *custom class loaders*—classes that subclass `java.lang.ClassLoader` (directly or indirectly). This is possible because the class loader class is not final, and developers are, in fact, encouraged to write their own class loaders specific to their individual needs.

Custom class loaders are represented as Java types, so they need to be loaded by a class loader, which is usually referred to as their *parent class loader*. This should not be confused with class inheritance and parent classes. Instead, class loaders are related by a form of *delegation*.

In figure 4.3, you can see the delegation hierarchy of class loaders and how the different loaders relate to each other. In some special cases, a custom class loader may have a different class loader as its parent, but the usual case is that it is the loading class loader.

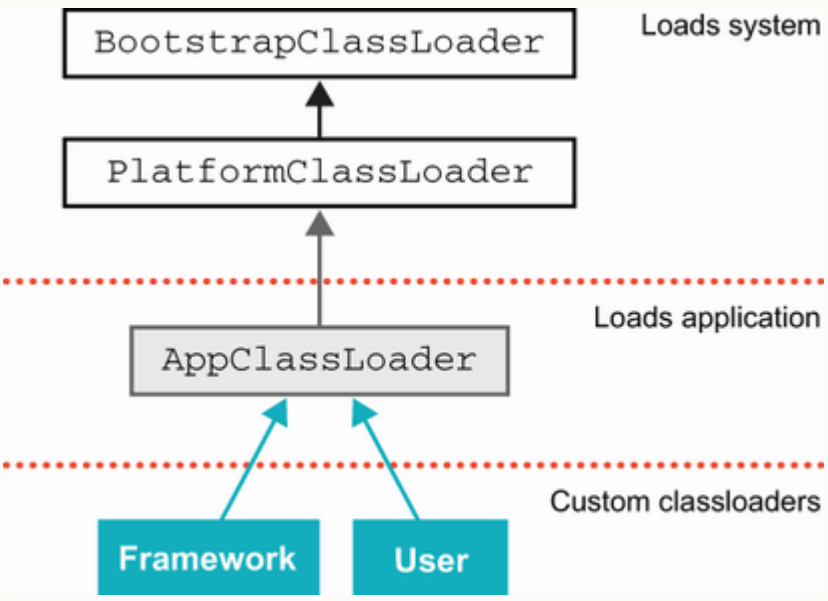


Figure 4.3 Classloader hierarchy

The key to the custom mechanism is found in the methods `loadClass()` and `findClass()`, which are defined on `ClassLoader`. The main entry point is `loadClass()` and a simplified form of the relevant code in `ClassLoader` follows:

```
protected Class<?> loadClass(String name, boolean resolve)
    throws ClassNotFoundException
  {
    synchronized (getClassLoadingLock(name)) {
      // First, check if the class has already been loaded
      Class<?> c = findLoadedClass(name);
      if (c == null) {
        // ...
        try {
          if (parent != null) {
            c = parent.loadClass(name, false);
          } else {
            c = findBootstrapClassOrNull(name);
          }
        } catch (ClassNotFoundException e) {
          // ClassNotFoundException thrown if class not found
          // from the non-null parent class loader
        }

        if (c == null) {
          // If still not found, then invoke findClass in order
          // to find the class.
          // ...
          c = findClass(name);

          // ...
        }
      }
      // ...

      return c;
    }
  }
```

Essentially, the `loadClass()` mechanism looks to see whether the class is already loaded and then asks its parent class loader. If that class loading fails (note the try-catch surrounding the call to `parent.loadClass(name, false)`), then the loading process delegates to `findClass()`. The definition of `findClass()` in `java.lang .ClassLoader` is very simple—it just throws a `ClassNotFoundException`.

At this point, let's return to a question that we posed at the start of the chapter and explore some of the exception and error types that can be encountered during class loading.

Class loading exceptions

The meaning of `ClassNotFoundException` is relatively simple: the class loader attempted to load the specified class but was unable to do so. That is, the class was unknown to the JVM at the point when loading was requested, and the JVM was unable to find it.

Next up is `NoClassDefFoundError`. Note that this is an *error* rather than an exception. This error indicates that the JVM did know of the existence of the requested class but did not find a definition for it in its internal metadata. Let's take a quick look at an example:

```
public class ExampleNoClassDef {

    public static class BadInit {
        private static int thisIsFine = 1 / 0;
    }

    public static void main(String[] args) {
        try {
            var init = new BadInit();
        } catch (Throwable t) {
            System.out.println(t);
        }
        var init2 = new BadInit();
        System.out.println(init2.thisIsFine);
    }
}
```

When this runs, we get some output like this:

```
$ java ExampleNoClassDef
java.lang.ExceptionInInitializerError
Exception in thread "main" java.lang.NoClassDefFoundError: Could
  not initialize class ExampleNoClassDef$BadInit
    at ExampleNoClassDef.main(ExampleNoClassDef.java:13)
```

This shows that the JVM tried to load the `BadInit` class but failed to do so. Nevertheless, the program caught the exception and tried to carry on. When the class was encountered for the second time, however, the JVM's internal metadata table showed that the class had been seen but that a valid class was not loaded.

The JVM effectively implements *negative caching* on a failed class loading attempt—the loading is not retried, and instead an error (`NoClassDefFoundError`) is thrown.

Another common error is `UnsupportedClassVersionError` , which is triggered when a class loading operation tries to load a class file that was compiled by a higher version of the Java source code compiler than the runtime supports. For example, consider a class compiled with Java 11 that we try to run on Java 8, shown next:

```
$ java ScratchImpl
Error: A JNI error has occurred please check your installation and try again
Exception in thread "main" java.lang.UnsupportedClassVersionError:
  ScratchImpl has been compiled by a more recent version of the Java
    Runtime (class file version 55.0), this version of the Java Runtime
    only recognizes class file versions up to 52.0
    at java.lang.ClassLoader.defineClass1(Native Method)
    at java.lang.ClassLoader.defineClass(ClassLoader.java:763)
    at java.security.SecureClassLoader.defineClass(SecureClassLoader.java:142)
    at java.net.URLClassLoader.defineClass(URLClassLoader.java:468)
    at java.net.URLClassLoader.access$100(URLClassLoader.java:74)
    at java.net.URLClassLoader$1.run(URLClassLoader.java:369)
    at java.net.URLClassLoader$1.run(URLClassLoader.java:363)
    at java.security.AccessController.doPrivileged(Native Method)
    at java.net.URLClassLoader.findClass(URLClassLoader.java:362)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:424)
    at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:349)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:357)
    at sun.launcher.LauncherHelper.checkAndLoadMain(LauncherHelper.java:495)
```

The Java 11 format bytecode may have features in it that are not supported by the runtime, so it is not safe to continue to try to load it. Note that because this is a Java 8 runtime, it does not have modular entries in the stack trace.

Finally, we should also mention `LinkageError` , which is the base class of a hierarchy containing `NoClassDefFoundError` , `VerifyError` , and `UnsatisfiedLinkError` , as well as several other possibilities.

The simplest form of custom class loading is simply to subclass `ClassLoader` and override `findClass()`. This allows us to reuse the `loadClass()` logic that we discussed earlier and to reduce the complexity in our class loader.

Our first example is the `SadClassLoader`, shown in the next code sample. It doesn't actually do anything, but it makes sure that you know that it was technically involved in the process and it wishes you well:

```
public class LoadSomeClasses {

   public static class SadClassloader extends ClassLoader {
      public SadClassloader() {
         super(SadClassloader.class.getClassLoader());
      }

      public Class<?> findClass(String name) throws
       ClassNotFoundException {
         System.out.println("I am very concerned that I
          couldn't find the class");
         throw new ClassNotFoundException(name);
      }
   }

   public static void main(String[] args) {
      if (args.length > 0) {
         var loader = new SadClassloader();
         for (var name : args) {
            System.out.println(name +" ::");
            try {
               var clazz = loader.loadClass(name);
               System.out.println(clazz);
            } catch (ClassNotFoundException x) {
               x.printStackTrace();
            }
         }
      }
   }
}
```

In our example, we set up a very simple class loader and some code that uses it to try to load classes that may or may not already be loaded.

NOTE One common convention for custom class loaders is to provide a no-argument constructor that calls the superclass constructor and provides the loading class loader as an argument (to become the parent).

Many custom class loaders are not that much more complex than our example—they just override `findClass()` to provide the specific capability that is needed. This could include, for example, looking for the class over the network. In one memorable case, a custom class loader loaded classes by connecting to a database via JDBC and accessing an encrypted binary column to get the bytes that would be used. This was to satisfy an encryption-at-rest requirement for very sensitive code in a highly regulated environment.

It is possible to do more than just override `findClass()`, however. For example, `loadClass()` is not final and so can be overridden, and, in fact, some custom class loaders do override it precisely to change the general logic we met earlier.

Finally, we also have the method `defineClass()` that is defined on `ClassLoader`. This method is key to class loading because it is the user-accessible method that performs the "loading and linking" process that we described earlier in the chapter. It takes an array of bytes and turns them into a class object. This is the primary mechanism that is used to load new classes at runtime that are not present on the classpath.

The call to `defineClass()` will work only if it is passed a buffer of bytes that are in the correct JVM class file format. If not, it will fail to load because either the loading or verification step will fail.

NOTE This method can be used for advanced techniques such as loading classes that are generated at runtime and that have no source code representation. This technique is how the lambda expressions mechanism works in Java. We will have more to say on this subject in chapter 17.

The `defineClass()` method is both protected and final and is defined on `java.lang.ClassLoader`, so it can be accessed only by subclasses of `ClassLoader`. Custom class loaders, therefore, always have access to the basic functionality of `defineClass()` but cannot tamper with the verification or other low-level class loading logic. This last point is important: being unable to change the verification algorithm is a very useful safety feature—a poorly written custom class loader cannot compromise the basic platform security the JVM provides.

In the case of the HotSpot virtual machine (which is by far the most common JVM implementation), `defineClass()` delegates to the native method `defineClass1()`, which does some basic checks and then calls a C function called `JVM_DefineClassWithSource()`.

This function is an entry point into the JVM, and it provides access into the C code of HotSpot. HotSpot uses the C `SystemDictionary` to load the new class via the C++ method `ClassFileParser::parseClassFile()`. This code actually runs much of the linking process, in particular, the verification algorithm.

Once class loading has completed, the bytecode of the methods is placed into HotSpot's metadata objects that represent the methods (they are called *methodOops*). They are then available for the bytecode interpreter to use. This can be thought of as a method cache conceptually, although the bytecode is actually held by the methodOops for performance reasons.

We have already met the `SadClassloader`. Now let's look at another couple of examples of custom class loaders, staring with a look at how class loading can be used to implement dependency injection.

Example: A dependency injection framework

We want to highlight the following two primary concepts that are highly relevant to DI:

- Units of functionality within a system have dependencies and configuration information upon which they rely for proper functioning.
- Many object systems have dependencies that are difficult or clumsy to express in code.

The picture you should have is of classes that contain behavior and configuration and dependencies that are external to the objects. This latter part is what is usually referred to as the *runtime wiring* of the objects. In this example, we'll discuss how a hypothetical DI framework could use class loaders to implement runtime wiring.

NOTE The approach we'll take is like a simplified version of the original implementation of the Spring framework. However, modern production DI frameworks have significantly higher complexity. Our example is for demonstration purposes only.

Let's start by looking at how we'd start an application under our imaginary DI framework, as shown here:

```
java -cp <CLASSPATH> org.wgjd.DIMain /path/to/config.xml
```

The class `DIMain` is the entry point class for the DI framework. It will read the config file, create the system of objects, and link them together ("wire them up"). Note that the

class `DIMain` is not an application class—it comes from the framework and is completely general.

We can also see that the `CLASSPATH` for the application must contain three things: a) the JAR files for the DI framework, b) the application classes that are referred to in the config.xml file, and c) any other (non-DI) dependencies that the application has. Let's look at an example config file, shown next:

```
<beans>

 <bean id="dao" class="app.ch04.PaymentsDAO">
  <constructor-arg index="0" value="jdbc:postgresql://db.wgjd.org/payments"/>
  <constructor-arg index="1" value="org.postgresql.Driver"/>
 </bean>

  <bean id="service" class="app.ch04.PaymentService">
   <constructor-arg index="0" ref="dao"/>
  </bean>

</beans>
```

The DI framework uses the config file to determine which objects to construct. This example needs to make the `dao` and `service` beans, and the framework will need to call the constructors for each bean, with the specified arguments.

Class loading occurs in two separate phases. The first phase (which is handled by the application class loader) loads the class `DIMain` and any framework classes that it refers to. Then `DIMain` starts to run and receives the location of the config file as a parameter to `main()`.

At this point, the framework is up and running in the JVM, but the user classes specified in `config.xml` haven't yet been touched. In fact, until `DIMain` examines the config file, the framework has no way of knowing what classes to load.

To bring up the application configuration specified in `config.xml`, a second phase of class loading is required. In our example, this uses a custom class loader.

First, the `config.xml` file is checked for consistency and to make sure it's error-free. Then, if all is well, the custom class loader tries to load the types from the `CLASSPATH`. If any of these fail, the whole process is aborted, causing a runtime error.

If this succeeds, the DI framework can proceed to instantiate the required objects in the correct order (with their constructor parameters). Finally, if all of this completes correctly, the application context is up and can begin to run.

It is worth reiterating that this example is hypothetical and illustrative. It would be entirely possible to build a simple DI framework that worked in the manner described here. However, the actual implementation of real DI systems is much more complicated in practice. Let's move on to look at another example.

Example: An instrumenting class loader

Consider a class loader that alters the bytecode of classes to add extra instrumentation information as they're loaded. When test cases are run against the transformed code, the instrumentation code records which methods and code branches are actually tested by the test cases. From this, the developer can see how thorough the unit tests for a class are.

This approach was the basis of the EMMA testing coverage tool, which is still available from **http://emma.sourceforge.net/**, although it is now rather outdated and has not been kept up-to-date for modern Java versions. Despite this, it's quite common to encounter frameworks and other code that use specialized class loaders that transform the bytecode as it's being loaded.

NOTE The technique of modifying bytecode as it is loaded is also seen in the *java agent* approach, which is used for performance monitoring, observability, and other goals by tools such as New Relic.

We've briefly touched on a couple of use cases for custom class loading. Many other areas of the Java technology space are big users of class loaders and related techniques. Some of the best-known examples follow:

- Plugin architectures
- Frameworks (whether vendor or homegrown)
- Class file retrieval from unusual locations (not filesystems or URLs)
- Java EE
- Any circumstance where new, unknown code may need to be added after the JVM process has already started running

Let's move on to discuss how the module system affects class loading and modifies the classic picture that we've just explained.

2.2 Modules and class loading

The modules system is designed to operate at a different level from class loading, which is a relatively low-level mechanism within the platform. Modules are about large-scale dependencies between program units, and class loading is about the small scale. However, it is important to understand how the two mechanisms intersect and the changes to program startup that have been caused by the arrival of modules.

Recall that when running on a modular JVM, to execute a program, the runtime will compute a module graph and try to satisfy it as a first step. This is referred to as *module resolution*, and it derives the transitive closure of the root module and its dependencies.

During this process, additional checks are performed (e.g., no modules with duplicate names, no split packages). The existence of the module graph means that fewer runtime class-loading problems are expected, because missing JARs on the module path can now be detected before the process even starts fully.

Beyond this, the modules system does not alter class loading much in most cases. There are some advanced possibilities (such as dynamically loading modular implementations of service provider interfaces by using reflection), but those are not likely to be encountered often by most developers.

4.3 Examining class files

Class files are binary blobs, so they aren't easy to work with directly. But there are many circumstances in which you'll find that investigating a class file is necessary.

Imagine that your application needs additional methods to be made public to allow better runtime monitoring (such as via JMX). The recompile and redeploy seems to complete fine, but when the management API is checked, the methods aren't there. Additional rebuild and redeploy steps have no effect.

To debug the deployment issue, you may need to check that `javac` has produced the class file that you think it has. Or you may need to investigate a class that you don't have source for and where you suspect the documentation is incorrect.

For these and similar tasks, you must make use of tools to examine the contents of class files. Fortunately, the standard Oracle JVM ships with a tool called `javap`, which is very handy for peeking inside and disassembling class files.

We'll start off by introducing `javap` and some of the basic switches it provides to examine aspects of class files. Then we'll discuss some of the representations for method names and types that the JVM uses internally. We'll move on to take a look at the constant pool—the JVM's "box of useful things"—which plays an important role in understanding how bytecode works.

3.1 Introducing javap

From seeing what methods a class declares to printing the bytecode, `javap` can be used for numerous useful tasks. Let's examine the simplest form of `javap` usage, as applied to the class-loading example from earlier in the chapter:

```
$ javap LoadSomeClasses.class
Compiled from "LoadSomeClasses.java"
public class LoadSomeClasses {
  public LoadSomeClasses();
  public static void main(java.lang.String[]);
}
```

The inner class has been compiled out into a separate class, so we need to also look at that one:

```
$ javap LoadSomeClasses\$SadClassloader.class
Compiled from "LoadSomeClasses.java"
public class LoadSomeClasses$SadClassloader extends java.lang.ClassLoader {
  public LoadSomeClasses$SadClassloader();
  public java.lang.Class<?> findClass(java.lang.String) throws
    java.lang.ClassNotFoundException;
}
```

By default, `javap` shows the public, protected, and default access (package-protected) visibility methods. The `-p` switch also shows the private methods and fields.

3.2 Internal form for method signatures

The JVM uses a slightly different form for method signatures internally than the human-readable form displayed by `javap`. As we delve deeper into the JVM, you'll see these internal names more frequently. If you're keen to keep going, you can jump ahead, but remember that this section's here—you may need to refer to it from later sections and chapters.

In the compact form, type names are compressed. For example, `int` is represented by `I`. These compact forms are sometimes referred to as *type descriptors*. A complete list is provided in table 4.1 (and includes `void`, which is not a type but does appear in method signatures).

Table 4.1 Type descriptors

Descriptor	Type
B	Byte
C	Char (a 16-bit Unicode character)
D	Double
F	Float
I	Int
J	Long
L<type name>;	Reference type (such as Ljava/lang/String; for a string)
S	Short
V	Void
Z	Boolean
[	Array-of

In some cases, the type descriptor can be longer than the type name that appears in source code (e.g., `Ljava/lang/Object;` is longer than `Object`, but the type descriptors are always fully qualified so they can be directly resolved).

`javap` provides a helpful switch, `-s`, which will output the type descriptors of signatures for you, so you don't have to work them out using the table. You can use a slightly

more advanced invocation of `javap` to show the signatures for some of the methods we looked at earlier, as shown next:

```
$ javap -s LoadSomeClasses.class
Compiled from "LoadSomeClasses.java"
public class LoadSomeClasses {
  public LoadSomeClasses();
    descriptor: ()V

  public static void main(java.lang.String[]);
    descriptor: ([Ljava/lang/String;)V
}
```

and for the inner class:

```
$ javap -s LoadSomeClasses\$SadClassloader.class
Compiled from "LoadSomeClasses.java"
public class LoadSomeClasses$SadClassloader extends java.lang.ClassLoader {
  public LoadSomeClasses$SadClassloader();
    descriptor: ()V

  public java.lang.Class<?> findClass(java.lang.String) throws
    java.lang.ClassNotFoundException;
    descriptor: (Ljava/lang/String;)Ljava/lang/Class;
}
```

As you can see, each type in a method signature is represented by a type descriptor.

In the next section, we'll see another use of type descriptors. This is in a very important part of the class file—the constant pool.

3.3 The constant pool

The constant pool is an area that provides handy shortcuts to other (constant) elements of the class file. If you've studied languages like C or Perl, which make explicit use of symbol tables, you can think of the constant pool as being a somewhat similar JVM concept.

Let's use a very simple example in the next listing to demonstrate the constant pool, so we don't swamp ourselves with detail. The next listing shows a simple "playpen" or

"scratchpad" class. This provides a way to quickly test out a Java syntax feature or library, by writing a small amount of code in `run()`.

Listing 4.1 Sample playpen class

```
package wgjd.ch04;

public class ScratchImpl {

   private static ScratchImpl inst = null;

   private ScratchImpl() {

   }

   private void run() {

   }

   public static void main(String[] args) {
      inst = new ScratchImpl();
      inst.run();
   }
}
```

To see the information in the constant pool, you can use `javap -v`. This prints a lot of additional information—much more than just the constant pool—but let's focus on the constant pool entries for the playpen, shown next:

```
#1 = Class #2 // wgjd/ch04/ScratchImpl

#2 = Utf8 wgjd/ch04/ScratchImpl

#3 = Class #4 // java/lang/Object

#4 = Utf8 java/lang/Object

#5 = Utf8 inst

#6 = Utf8 Lwgjd/ch04/ScratchImpl;

#7 = Utf8 <clinit>
```

#8 = Utf8 ()V

#9 = Utf8 Code

#10 = Fieldref #1.#11 // wgjd/ch04/ScratchImpl.inst:Lwgjd/ch04/ScratchImpl;

#11 = NameAndType #5:#6 // instance:Lwgjd/ch04/ScratchImpl;

#12 = Utf8 LineNumberTable

#13 = Utf8 LocalVariableTable

#14 = Utf8 <init>

#15 = Methodref #3.#16 // java/lang/Object."<init>":()V

#16 = NameAndType #14:#8 // "<init>":()V

#17 = Utf8 this

#18 = Utf8 run

#19 = Utf8 ([Ljava/lang/String;)V

#20 = Methodref #1.#21 // wgjd/ch04/ScratchImpl.run:()V

#21 = NameAndType #18:#8 // run:()V

#22 = Utf8 args

#23 = Utf8 [Ljava/lang/String;

#24 = Utf8 main

#25 = Methodref #1.#16 // wgjd/ch04/ScratchImpl."<init>":()V

#26 = Methodref #1.#27 // wgjd/ch04/ScratchImpl.run:([Ljava/lang/String;)V

#27 = NameAndType #18:#19 // run:([Ljava/lang/String;)V

#28 = Utf8 SourceFile

#29 = Utf8 ScratchImpl.java

As you can see, constant pool entries are typed. They also refer to each other, so, for example, an entry of type `Class` will refer to an entry of type `Utf8`. A `Utf8` entry means a string, so the `Utf8` entry that a `Class` entry points out will be the name of the class.

Table 4.2 shows the set of possibilities for entries in the constant pool. Entries from the constant pool are sometimes discussed with a `CONSTANT_` prefix, such as `CONSTANT_Class`. This is to make it clear that they are not Java types, in situations where they could be confused.

Table 4.2 Constant pool entries

Name	Description
`Class`	A class constant. Points at the name of the class (as a `Utf8` entry).
Fieldref	Defines a field. Points at the `Class` and `NameAndType` of this field.
`Methodref`	Defines a method. Points at the `Class` and `NameAndType` of this field.
InterfaceMethodref	Defines an interface method. Points at the `Class` and `NameAndType` of this field.
`String`	A string constant. Points at the `Utf8` entry that holds the characters.
Integer	An integer constant (4 bytes).
`Float`	A floating-point constant (4 bytes).
Long	A long constant (8 bytes).
`Double`	A double-precision floating-point constant (8 bytes).
NameAndType	Describes a name and type pair. The type points at the `Utf8` that holds the type descriptor for the type.
`Utf8`	A stream of bytes representing `Utf8`-encoded characters.
InvokeDynamic	Part of invokedynamic mechanism—see chapter 17.

Name	Description
`MethodHandle`	Part of `invokedynamic` mechanism—see chapter 17.
MethodType	Part of invokedynamic mechanism—see chapter 17.

Using this table, you can look at an example constant resolution from the constant pool of the playpen. Consider the `Fieldref` at entry #10. To resolve a field, you need a name, a type, and a class where it resides: #10 has the value #1.#11, which means constant #11 from class #1. It's easy to check that #1 is indeed a constant of type `Class`, and #11 is a `NameAndType`. #1 refers to the `ScratchImpl` Java class itself, and #11 refers to #5:#6—a variable called `inst` of type `ScratchImpl`. So, overall, #10 refers to the static variable inst in the `ScratchImpl` class itself (which you might have been able to guess from the output above).

In the verification step of class loading, there's a step to check that the static information in the class file is consistent. The preceding example shows the kind of integrity check that the runtime will perform when loading a new class.

We've discussed some of the basic anatomy of a class file. Let's move on to the next topic, where we'll delve into the world of bytecode. Understanding how source code is turned into bytecode will help you gain a better understanding of how your code will run. In turn, this will lead to more insights into the platform's capabilities when we reach chapter 6 and beyond.

4.4 Bytecode

Bytecode has been a somewhat behind-the-scenes player in our discussion so far. Let's start by reviewing what we've already learned about it:

- Bytecode is an intermediate representation of a program, halfway between human readable source and machine code.
- Bytecode is produced by `javac` from Java source code files.
- Some high-level language features have been compiled away and don't appear in bytecode. For example, Java's looping constructs (`for`, `while`, and the like) are gone, turned into bytecode branch instructions.
- Each opcode is represented by a single byte (hence the name *bytecode*).

- Bytecode is an abstract representation, not “machine code for an imaginary CPU.”
- Bytecode can be further compiled to machine code, usually “just in time.”

When explaining bytecode, there can be a slight chicken-and-egg problem. To fully understand what's going on, you need to understand both bytecode and the runtime environment that it executes in. This is a rather circular dependency, so to solve it, we'll start by diving in and looking at a relatively simple example. Even if you don't understand everything that's in this example on the first pass, you can come back to it after you've read more about bytecode in the following sections.

After the example, we'll provide some context about the runtime environment, and then catalogue the JVM's opcodes, including bytecodes for arithmetic, invocation, shortcut forms, and more. At the end, we'll round off with another example, based on string concatenation. Let's get started by looking at how you can examine bytecode from a .class file.

4.1 Disassembling a class

Using `javap` with the `-c` switch, you can disassemble classes. In our example, we'll use the `ScratchImpl` class we met earlier. The main focus will be to examine the bytecode that makes up methods. We'll also use the `-p` switch so we can see bytecode from private methods.

Let's work section by section—there's a lot of information in each part of `javap`'s output, and it's easy to become overwhelmed. First, the header. There's nothing terribly unexpected or exciting in here, as shown here:

```
$ javap -c -p wgjd/ch04/ScratchImpl.class

Compiled from "ScratchImpl.java"

public class wgjd.ch04.ScratchImpl extends java.lang.Object {
  private static wgjd.ch04.ScratchImpl inst;
```

Next is the static block. This is where variable initialization is placed, so this represents initializing `inst` to `null`. The keen-eyed reader might guess that `putstatic` could be a bytecode that puts a value in a static field:

```
static {};

Code:
```

```
  0: aconst_null
  1: putstatic #10 // Field inst:Lwgjd/ch04/ScratchImpl;
  4: return
```

The numbers in the preceding code represent the offset into the bytecode stream from the start of the method. So byte 1 is the `putstatic` opcode, and bytes 2 and 3 represent a 16-bit index into the constant pool. In this case, the 16-bit index is the value 10, which means that the value (in this case, `null`) will be stored in the field indicated by constant pool entry #10. Byte 4 from the start of the bytecode stream is the return opcode—the end of the block of code.

Next up is the constructor:

```
  private wgjd.ch04.ScratchImpl();

  Code:
    0: aload_0
    1: invokespecial #15 // Method java/lang/Object."<init>":()V
    4: return
```

Remember that in Java, the void constructor will always implicitly call the superclass constructor. Here you can see this in the bytecode—it's the `invokespecial` instruction. In general, any method call will be turned into one of the JVM's five `invoke` instructions, which we'll meet in section 4.4.7.

The constructor invocation requires a target, which is provided by the `aload_0` instruction. This loads a reference (an Address) and uses a shortcut form (which we'll meet properly in section 4.4.9) to load the 0th local variable, which is just `this`, the current object.

There's basically no code in the `run()` method, because this is just a scratchpad class for testing out code. This method immediately returns to the caller and does not pass a value back (which is correct, because the method returns `void`):

```
  private void run();

  Code:
    0: return
```

In the main method, we initialize `inst` and do a bit of object creation. This demonstrates some very common basic bytecode patterns that we can learn to recognize:

```
public static void main(java.lang.String[]);

Code:
  0: new #1 // class wgjd/ch04/ScratchImpl
  3: dup
  4: invokespecial #21 // Method "<init>":()V
```

This pattern of three bytecode instructions— `new`, `dup`, and `invokespecial` of a method called `<init>` —always represents the creation of a new instance.

The `new` opcode allocates memory for a new instance and places a reference to it on the top of the stack. The `dup` opcode duplicates the reference that's on top of the stack (so now there are two copies). To finish fully creating the object, we need to call the body of the constructor. The `<init>` method contains the code for the constructor body, so we call that code block with `invokespecial`.

When methods are called, the reference to the receiver object (if any) is consumed from the stack, along with any arguments to the method. This is why we need to perform a `dup` first—without it, the newly allocated object will have its only reference consumed by the `invoke` and will be inaccessible after this point.

Let's look at the remaining bytecodes for the main method:

```
 7: putstatic #10 // Field inst:Lwgjd/ch04/ScratchImpl;
10: getstatic #10 // Field inst:Lwgjd/ch04/ScratchImpl;
13: invokevirtual #22 // Method run:()V
16: return
```

Instruction 7 saves the address of the singleton instance that has been created. Instruction 10 puts it back on top of the stack, so that instruction 13 can call a method on it. This is done with the `invokevirtual` opcode, which carries out Java's "standard" dispatch for instance methods.

NOTE In general, the bytecode produced by `javac` is a simple representation—it isn't highly optimized. The overall strategy is that just-in-time (JIT) compilers do a lot of optimizing, so it helps if they have a relatively plain and simple starting point. The expres-

sion, "Bytecode should be dumb," describes the general feeling of JVM implementers toward the bytecode produced from source languages.

The `invokevirtual` opcode includes checking for overrides of the method in the object's inheritance hierarchy. You might notice that this is a bit odd, because private methods can't be overridden. You might guess that the source code compiler could actually emit `invokespecial` instead of `invokevirtual` for private methods. In fact, this used to be the case and was changed only in recent versions of Java. For details, see the section on nestmates in chapter 17.

Let's move on to discuss the runtime environment that bytecode needs. After that, we'll introduce the tables that we'll use to describe the major families of bytecode instructions—load/store, arithmetic, execution control, method invocation, and platform operations. Then we'll discuss possible shortcut forms of opcodes, before moving on to another example.

4.2 The runtime environment

Understanding the operation of the stack machine that the JVM uses is critical to understanding bytecode. One of the most obvious ways that the JVM doesn't look like a hardware CPU (such as an x64 or ARM chip) is that the JVM doesn't have processor registers and instead uses a stack for all calculations and operations. This is referred to as the *evaluation stack* (it's officially called the *operand stack* in the VM specification, and we'll use the two terms interchangeably).

The evaluation stack is local to a method, and when a method is called, a fresh evaluation stack is created. Of course, the JVM also has a *call stack* for each Java thread that records which methods have been executed (and which forms the basis of stack traces in Java). It's important to keep the distinction between the per-thread call stack and the per-method evaluation stack clear.

Figure 4.4 shows how the evaluation stack might be used to perform an addition operation on two int constants. We're showing the equivalent JVM bytecode below each step—we'll meet this bytecode later in the chapter, so don't worry if it doesn't make complete sense right now.

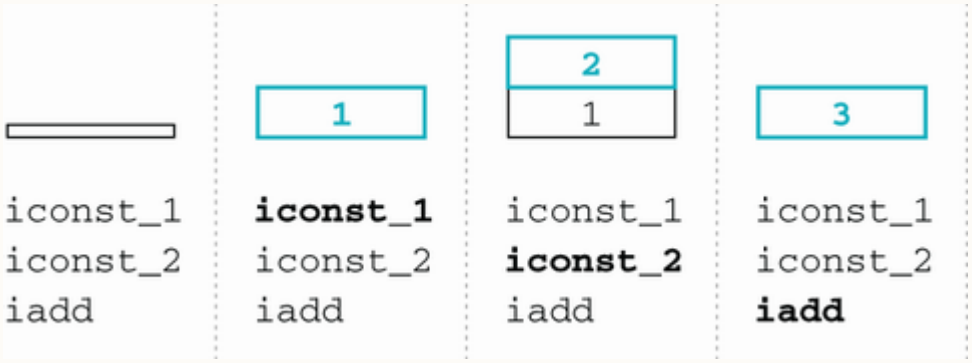


Figure 4.4 Using a stack for numerical calculations

As we discussed earlier in this chapter, when a class is linked into the running environment, its bytecode will be checked, and a lot of that verification boils down to analyzing the pattern of types on the stack.

NOTE Manipulations of the values on the stack work only if the values on the stack have the correct types. Undefined or bad things could happen if, for example, we pushed a reference to an object onto the stack and then tried to treat it as an int and do arithmetic on it.

The verification phase of class loading performs extensive checks to ensure that methods in newly loaded classes don't try to abuse the stack. This prevents a malformed (or deliberately evil) class from ever being accepted by the system and causing problems.

As a method runs, it needs an area of memory to use as an evaluation stack, for computing new values. In addition, every running thread needs a call stack that records which methods are currently in flight (the stack that would be reported by a stack trace). These two stacks will interact in some cases. Consider this bit of code:

```
var numPets = 3 + petRecords.getNumberOfPets("Ben");
```

To evaluate this, the JVM puts `3` on the operand stack. Then it needs to call a method to calculate how many pets Ben has. To do this, it pushes the receiver object (the one the method is being called on—`petRecords`, in this example) onto the evaluation stack, followed by any call arguments.

Then the `getNumberOfPets()` method is called using one of the `invoke` opcodes, which will cause control to transfer to the called method and the just-entered method to appear in the call stack. But, as the JVM enters the new method, it starts using a fresh operand stack, so the values already on the caller's operand stack can't possibly affect results calculated in the called method.

When `getNumberOfPets()` completes, the return value is placed onto the operand stack of the caller, as part of the process whereby `getNumberOfPets()` is removed from the call stack. Then the addition operation takes the two values and adds them.

Let's now turn to examining bytecode. This is a large subject, with lots of special cases, so we're going to present an overview of the main features rather than a complete treatment.

4.3 Introduction to opcodes

JVM bytecode consists of a sequence of operation codes (opcodes), possibly with some arguments following each instruction. Opcodes expect to find the stack in a given state and transform the stack, so that the arguments are removed and results placed there instead.

Each opcode is denoted by a single-byte value, so at most 255 possible opcodes exist. Currently, only around 200 are used. This is too many for us to list exhaustively (but a complete list can be found at **http://mng.bz/aJaX**). Fortunately, most opcodes fit into one of a number of basic families that provide similar functionality. We'll discuss each family in turn, to help you get a feel for them. Some operations don't fit cleanly into any of the families, but they tend to be encountered less often.

NOTE The JVM isn't a purely object-oriented runtime environment. It has knowledge of primitive types. This shows up in some of the opcode families—some basic opcode types (such as `store` and `add`) are required to have a number of variations that differ, depending on the primitive type they're acting upon.

The opcode tables have the following four columns:

- *Name*—This is a general name for the type of opcode. In many cases, several related opcodes do similar things.
- *Args*—The arguments that the opcode takes. Arguments that start with `i` are (unsigned) bytes that are used to form a lookup index in the constant pool or local variable table.

NOTE To make longer indices, bytes are joined together, so that `i1, i2` means "make a 16-bit index out of these two bytes" via bit shifting and addition: `((i1 << 8) + i2)`

If an arg is shown in brackets, it means that not all forms of the opcode will use it.

- *Stack layout*—This shows the state of the stack before and after the opcode has executed. Elements in brackets indicate that not all forms of the opcode use them or that the elements are optional (such as for invocation opcodes).
- *Description*—What the opcode does.

Let's look at an example of a row from table 4.3 by examining the entry for the `getfield` opcode. This is used to read a value from a field of an object.

getfield	i1, i2	[obj] -> [val]	Gets the field at the constant pool index specified from the object on top of the stack.

The first column gives the name of the opcode—`getfield`. The next column says that there are two arguments that follow the opcode in the bytecode stream. These arguments are put together to make a 16-bit value that is looked up in the constant pool to see which field is wanted (remember that constant pool indexes are always 16-bit). The stack layout column shows that the reference to the object is replaced by the value of the field.

This pattern of removing object instances as part of the operation is just a way to make bytecode compact, without lots of tedious cleanup and having to remember to remove object instances that you're finished with.

4.4 Load and store opcodes

The family of load and store opcodes is concerned with loading values onto the stack or retrieving them. Table 4.3 shows the main operations in the load/store family.

Table 4.3 Load and store opcodes

Name	Args	Stack layout	Description
load	(i1)	[] -> [val]	Loads a value (primitive or reference) from a local variable onto the stack. Has shortcut forms and type-specific variants.
ldc	i1	[] -> [val]	Loads a constant from the pool onto the stack. Has type-specific and wide variants.
store	(i1)	[val] -> []	Stores a value (primitive or reference) in a local variable, removing it from the stack in the process. Has shortcut forms and type-specific variants.
dup		[val] -> [val, val]	Duplicates the value on top of the stack. Has variant forms.
get-field	i1, i2	[obj] -> [val]	Gets the field at the constant pool index specified from the object on top of the stack.
put-field	i1, i2	[obj, val] -> []	Puts the value into the object's field at the specified constant pool index.
get-static	i1, i2	[] -> [val]	Gets the value of the static field at the constant pool index specified.
put-static	i1, i2	[val] -> []	Puts the value into the static field at the specified constant pool index.

As we noted earlier, a number of different forms of the load and store instructions exist. For example, a `dload` opcode loads a double onto the stack from a local variable, and an `astore` opcode pops an object reference off the stack and into a local variable.

Let's do a quick example of `getfield` and `putfield`. This simple class:

```
public class Scratch {
   private int i;

   public Scratch() {
      i = 0;
   }

   public int getI() {
      return i;
   }

   public void setI(int i) {
      this.i = i;
   }
}
```

will decompile the getter and setter as:

```
public int getI();
   Code:
      0: aload_0
      1: getfield      #7                  // Field i:I
      4: ireturn

  public void setI(int);
   Code:
      0: aload_0
      1: iload_1
      2: putfield      #7                  // Field i:I
      5: return
```

which shows how the stack is used to hold temporary variables before transferring them to heap storage.

4.4.5 Arithmetic opcodes

These opcodes perform arithmetic on the stack. They take arguments from the top of the stack and perform the required calculation on them. The arguments (which are always primitive types) must always match exactly, but the platform provides a wealth of op-codes to cast one primitive type to another. Table 4.4 shows the basic arithmetic operations.

Table 4.4 Arithmetic opcodes

Name	Args	Stack layout	Description
add		[val1, val2] -> [res]	Adds two values (which must be of the same primitive type) from the top of the stack and stores the result on the stack. Has shortcut forms and type-specific variants.
sub		[val1, val2] -> [res]	Subtracts two values (of the same primitive type) from the top of the stack. Has shortcut forms and type-specific variants.
div		[val1, val2] -> [res]	Divides two values (of the same primitive type) from the top of the stack. Has shortcut forms and type-specific variants.
mul		[val1, val2] -> [res]	Multiplies two values (of the same primitive type) from top of the stack. Has shortcut forms and type-specific variants.
(cast)		[value] -> [res]	Casts a value from one primitive type to another. Has forms corresponding to each possible cast.

The cast opcodes have very short names, such as `i2d` for an `int` to `double` cast. In particular, the word *cast* doesn't appear in the names, which is why it's in parentheses in the table.

4.4.6 Execution flow control opcodes

As mentioned earlier, the control constructs of high-level languages aren't present in JVM bytecode. Instead, flow control is handled by a small number of primitives, which are shown in table 4.5.

Table 4.5 Execution control opcodes

Name	Args	Stack layout	Description
if	b1, b2	[val1, val2] -> [] or [val1] -> []	If the specific condition matches, jump to the specified branch offset.
goto	b1, b2	[] -> []	Unconditionally jump to the branch offset. Has wide form.
tableswitch	{depends}	[index] -> []	Used to implement switch.
lookupswitch	{depends}	[key] -> []	Used to implement switch.

Like the index bytes used to look up constants, the `b1, b2` args are used to construct a bytecode location within this method to jump to. They cannot be used to jump outside of the method—this is checked at class-loading time and would cause the class to fail verification.

The family of `if` opcodes is a little larger than you might expect—it has more than 15 instructions to handle the various source code possibilities (e.g., numeric comparison, reference equality).

NOTE The family of `if` opcodes also contains two deprecated instructions, `jsr` and `ret`, which are no longer produced by `javac` and are illegal in modern Java versions.

A wide form of the `goto` instruction (`goto_w`) takes 4 bytes of arguments and constructs an offset, which can be larger than 64 KB. This isn't often needed because it would only apply to very, very large methods (and such methods have other problems, such as being too large to be JIT compiled). There is also `ldc_w`, which can be used to address very large constant pools.

4.4.7 Invocation opcodes

The invocation opcodes comprise four opcodes for handling general method calling, plus the unusual `invokedynamic` opcode, which was added in Java 7. We'll discuss this spe-

cial case in more detail in chapter 17. The five method invocation opcodes are shown in table 4.6.

Table 4.6 Invocation opcodes

Name	Args	Stack layout	Description
invokestatic	i1, i2	[(val1, ...)] -> []	Calls a static method.
invokevirtual	i1, i2	[obj, (val1, ...)] -> []	Calls a "normal" instance method.
invokeinterface	i1, i2, count, 0	[obj, (val1, ...)] -> []	Calls an interface method.
invokespecial	i1, i2	[obj, (val1, ...)] -> []	Calls a "special" instance method, such as a constructor.
invokedynamic	i1, i2, 0, 0	[val1, ...] -> []	Dynamic invocation; see chapter 17.

It's easiest to see the difference between these opcodes with an extended example, shown here:

```
long time = System.currentTimeMillis();

// This explicit typing is deliberate... read on
HashMap<String, String> hm = new HashMap<>();
hm.put("now", "bar");

Map<String, String> m = hm;
m.put("foo", "baz");
```

Let's use `javap -c` to look at the bytecode for this:

```
Code:
     0: invokestatic  #2  // Method java/lang/System.currentTimeMillis:()J
     3: lstore_1
     4: new           #3 // class java/util/HashMap
```

```
 7: dup
 8: invokespecial #4 // Method java/util/HashMap."<init>":()V
11: astore_3
12: aload_3
13: ldc          #5 // String now
15: ldc          #6 // String bar
17: invokevirtual #7 // Method java/util/HashMap.put:(
                     //Ljava/lang/Object;Ljava/lang/Object;)
                     //Ljava/lang/Object;
20: pop
21: aload_3
22: astore       4
24: aload        4
26: ldc          #8 // String foo
28: ldc          #9 // String baz
30: invokeinterface #10,  3 // InterfaceMethod java/util/Map.put:(
                        //Ljava/lang/Object;Ljava/lang/Object;)
                        //Ljava/lang/Object;
35: pop
```

As we discussed earlier, the Java method calls are actually turned into one of several possible `invoke*` bytecodes. Let's take a closer look:

```
0: invokestatic  #2 // Method java/lang/System.currentTimeMillis:()J
3: lstore_1
```

The static call to `System.currentTimeMillis()` is turned into an `invokestatic` that appears at position 0 in the bytecode. This method takes no parameters, so nothing needs to be loaded onto the evaluation stack before the call is dispatched.

Next, the two bytes `00 02` appear in the byte stream. These are combined into a 16-bit number that is used as an offset into the constant pool.

The decompiler helpfully includes a comment that lets the user know which method offset `#2` corresponds to. In this case, as expected, it's the method `System.currentTimeMillis()`.

On return, the result of the call is placed on the stack, and at offset 3, we see the single, argument-less opcode `lstore_1` that saves this return value off into the local variable 1.

Human readers are, of course, able to see that the variable `time` is never used again. However, one of the design goals of `javac` is to represent the contents of the Java

source code as faithfully as possible, whether or not it makes sense. Therefore, the return value of `System.currentTimeMillis()` is stored, even though it is not used after this point in the program.

This is "dumb bytecode" in action: remember that from the point of view of the platform, the class file format is the input format to the compiler that really matters—the JIT compiler:

```
 4: new          #3 // class java/util/HashMap
 7: dup
 8: invokespecial #4 // Method java/util/HashMap."<init>":()V
11: astore_3
12: aload_3
13: ldc          #5 // String now
15: ldc          #6 // String bar
17: invokevirtual #7 // Method java/util/HashMap.put:(
                     //Ljava/lang/Object;Ljava/lang/Object;)
                     //Ljava/lang/Object;
20: pop
```

Bytecodes 4 to 10 create a new `HashMap` instance, before instruction 11 saves a copy of it into a local variable. Next, instructions 12 to 16 set up the stack with the `HashMap` object and the arguments for the call to `put()`. The actual invocation of the `put()` method is performed by instructions 17 to 19.

The invoke opcode used this time is `invokevirtual` because the static type of the local variable was declared as `HashMap`—a class type. We will see what will happen if the local variable is declared as `Map` in a moment.

An instance method call differs from a static method call because a static call does not have an instance on which the method is called (sometimes called the receiver object).

NOTE In bytecode, an instance call must be set up by placing the receiver and any call arguments on the evaluation stack and then issuing the invoke instruction.

In this case, the return value from `put()` is not used, so instruction 20 discards it, as shown here:

```
21: aload_3
22: astore       4
24: aload        4
```

```
26: ldc           #8 // String foo
28: ldc           #9 // String baz
30: invokeinterface #10,  3 //InterfaceMethod java/util/Map.put:(
                      //Ljava/lang/Object;Ljava/lang/Object;)
                      //Ljava/lang/Object;
35: pop
```

The sequence of bytes from 21 to 25 seems rather odd at first glance. The `HashMap` instance that we created at 4 and saved to local variable 3 at instruction 11 is now loaded back onto the stack, and a copy of the reference is saved to local variable 4. This process removes it from the stack, so it must be reloaded (from variable 4) before use. This shuffling occurs because in the original Java code, we created an additional local variable (of type `Map` rather than `HashMap`), even though it always refers to the same object as the original variable. This is another example of the bytecode staying as close as possible to the original source code.

After the stack and variable shuffling, the values to be placed in the map are loaded at instructions 26 to 29. With the stack prepared with receiver and arguments, the call to `put()` is dispatched at instruction 30. This time, the opcode is `invokeinterface`, even though the exact same method is actually being called. This is because the Java local variable is of type `Map`—an interface type. Once again, the return value from `put()` is discarded, via the `pop` at instruction 35.

As well as knowing which Java method invocations turn into which operations, you should notice a couple of other wrinkles about the invocation opcodes. First off is that `invokeinterface` has extra parameters. These are present for historical and backward compatibility reasons and aren't used these days. The two extra zeros on `invokedynamic` are present for forward-compatibility reasons.

The other important point is the distinction between a regular and a special instance method call. A regular call is *virtual*, which means that the exact method to be called is looked up at runtime using the standard Java rules of method overriding.

However, a couple of special cases exist, including calls to a superclass method. In these cases, you don't want the override rules to be triggered, so you need a different invocation opcode to allow for this case. This is why the opcode set needs an opcode for invocation of methods without the override mechanism—`invokespecial`—which instead indicates exactly which method will be called.

4.8 Platform operation opcodes

The platform operation family of opcodes includes the new opcode, for allocating new object instances, and the thread-related opcodes, such as `monitorenter` and `monitorexit`. The details of this family can be seen in table 4.7.

Table 4.7 Platform opcodes

Name	Args	Stack layout	Description
new	i1, i2	[] -> [obj]	Allocates memory for a new object, of the type specified by the constant at the specified index.
monitorenter		[obj] -> []	Locks an object. See chapter 5.
monitorexit		[obj] -> []	Unlocks an object. See chapter 5.

The platform opcodes are used to control certain aspects of the object lifecycle, such as creating new objects and locking them. It's important to notice that the new opcode allocates only storage. The high-level conception of object construction also includes running the code inside the constructor.

At the bytecode level, the constructor is turned into a method with a special name—`<init>`. This can't be called from user Java code, but it can be called by bytecode. This leads to the distinctive bytecode pattern that directly corresponds to object creation—a `new` followed by a `dup` followed by an `invokespecial` to call the `<init>` method, as we saw earlier.

The `monitorenter` and `monitorexit` bytecodes correspond to the start and end of a synchronized block.

4.9 Shortcut opcode forms

Many of the opcodes have shortcut forms to save a few bytes here and there. The general pattern is that certain local variables will be accessed much more frequently than others, so it makes sense to have a special opcode that means "do the general operation directly on the local variable" rather than having to specify the local variable as an argu-

ment. This gives rise to opcodes such as `aload_0` and `dstore_2` within the load/store family, which are 1 byte shorter than the equivalent byte sequences, `aload 00` or `dstore 02`.

NOTE One byte saved may not sound like much, but it adds up over the entire class. Java's original use case was applets, which were often downloaded over dial-up modems, at speeds of 28.8 *kilobits* per second. With that speed of bandwidth, it was important to save bytes wherever possible.

To become a truly well-grounded Java developer, you should run `javap` against some of your own classes and learn to recognize common bytecode patterns. For now, with this brief introduction to bytecode under our belts, let's move on to tackle our next subject—reflection.

4.5 Reflection

One of the key techniques that a well-grounded Java developer should have at their command is *reflection*. This is an extremely powerful capability, but many developers struggle with it at first because it seems alien to the way that most Java developers think about code.

Reflection is the ability to query or *introspect* objects and discover (and use) their capabilities at runtime. It can be thought of as several different things, depending on context:

- A programming language API
- A programming style or technique
- A runtime mechanism that enables the technique
- A property of the language type system

Reflection in an object-oriented system is essentially the idea that the programming environment can represent the types and methods of the program as objects. This is possible only in languages that have a runtime that supports this, and it is a fundamentally dynamic aspect of a language.

When using the reflective style of programming, it is possible to manipulate objects without using their static types at all. This seems like a step backward, but if we can work with objects without needing to know their static types, then it means that we can build libraries, frameworks, and tools that can work with *any* type—including types that did not even exist when our code was written.

When Java was a young language, reflection was one of the key technological innovations that it brought to the mainstream. Although other languages (notably Smalltalk) had introduced it much earlier, it was not a common part of many languages at the time Java was released.

15.1 Introducing reflection

The abstract description of reflection can often seem confusing or hard to grasp. Let's look at some simple examples in JShell to try to get a more concrete view of what reflection is:

```
jshell> Object o = new Object();
o ==> java.lang.Object@a67c67e

jshell> Class<?> clz = o.getClass();
clz ==> class java.lang.Object
```

This is our first glimpse of reflection—a class object for the type `Object`. In fact, the actual type of `clz` is `Class<Object>`, but when we obtain a class object from class loading or `getClass()`, we have to handle it using the unknown type, `?`, in the generics, as follows:

```
jshell> Class<Object> clz = Object.class;
clz ==> class java.lang.Object

jshell> Class<Object> clz = o.getClass();
|  Error:
|  incompatible types: java.lang.Class<capture#1 of ? extends
   java.lang.Object> cannot be converted to java.lang.Class<java.lang.Object>
|  Class<Object> clz = o.getClass();
|                      ^----------^
```

This is because reflection is a dynamic, runtime mechanism, and the true type `Class<Object>` is not known to the source code compiler. This process introduces irreducible extra complexity to working with reflection because we cannot rely on the Java type system to help us very much. On the other hand, this dynamic nature is the key point of reflection—if we don't know what type something is at compile time and have to treat it in a very general way, we can exploit this flexibility to build an open, extensible system.

NOTE Reflection produces a fundamentally open system, and as we saw in chapter 2, this can come into conflict with the more encapsulated systems that Java modules try to bring to the platform.

Many familiar frameworks and developer tools rely heavily on reflection to achieve their capabilities, such as debuggers and code browsers. Plugin architectures, interactive environments, and REPLs also use reflection extensively. In fact, JShell itself could not be built in a language without a reflection subsystem. Let's exploit this and use JShell to explore some of reflection's key features, as shown next:

```
jshell> class Pet {
   ...>   public void feed() {
   ...>     System.out.println("Feed the pet");
   ...>   }
   ...> }
|  created class Pet

jshell> var clz = Pet.class;
clz ==> class Pet
```

Now we have an object that represents the class type of `Pet` that we can use to do other actions, such as creating a new instance, as follows:

```
jshell> Object o = clz.newInstance();
o ==> Pet@66480dd7
```

The problem we have is that `newInstance()` returns `Object`, which isn't a very useful type. We could, of course, cast `o` back to `Pet`, but this requires us to know ahead of time what types we're working with, which rather defeats the point of the dynamic nature of reflection. So let's try something else:

```
jshell> import java.lang.reflect.Method;

jshell> Method m = clz.getMethod("feed", new Class[0]);
m ==> public void Pet.feed()
```

Now we have an object that represents the method `feed()`, but it represents it as abstract metadata—it is not attached to any specific instance.

The natural thing to do with an object that represents a method is to call it. The class `java.lang.reflect.Method` defines a method `invoke()` that has the effect of calling the method that the `Method` object represents.

NOTE When working in JShell, we avoid a lot of exception-handling code. When writing regular Java code that uses reflection, you will have to deal with the possible exception types in one way or another.

For this call to succeed, we must provide the right number and types of arguments. This argument list must include the *receiver object* on which the method is being called reflectively (assuming the method is an instance method). In our simple example, this looks like this:

```
jshell> Object ret = m.invoke(o);
Feed the pet
ret ==> null                    (1)
```

(1) The call returns null because the feed() method is actually void.

As well as the `Method` objects, reflection also provides for objects that represent other fundamental concepts within the Java type system and language, such as fields, annotations, and constructors. These classes are found in the `java.lang.reflect` package, and some of them (such as `Constructor`) are generic types.

The reflection subsystem also had to be upgraded to deal with modules. Just as classes and methods can be treated reflectively, so there needs to be a reflective API for working with modules. The key class is, perhaps unsurprisingly, `java.lang.Module`, and we can access directly from a `Class` object as follows:

```
var module = String.class.getModule();
var descriptor = module.getDescriptor();
```

The descriptor of a module is of type `ModuleDescriptor` and provides a read-only view of the metadata about a module—basically equivalent to the contents of `module-info.class`.

Dynamic capabilities, such as discovery of modules, are also possible in the new reflective API. This is achieved via interfaces such as `ModuleFinder`, but a detailed description of how to work reflectively with the modules system is outside the scope of this book—

the interested reader should consult chapter 12 of Nicolai Parlog's book, *The Java Module System* (Manning, 2019), **http://mng.bz/gwGG**.

5.2 Combining class loading and reflection

Let's look at an example that combines class loading and reflection. We won't need a full class loader that obeys the usual `findClass()` and `loadClass()` protocols. Instead, we'll just subclass `ClassLoader` to gain access to the protected `defineClass()` method.

The main method takes a list of filenames, and, if they're a Java class, it uses reflection to access each method in turn and detect whether or not it's a native method, as shown next:

```
public class NativeMethodChecker {

   public static class EasyLoader extends ClassLoader {
      public EasyLoader() {
         super(EasyLoader.class.getClassLoader());
      }

      public Class<?> loadFromDisk(String fName) throws IOException {
         var b = Files.readAllBytes(Path.of(fName));
         return defineClass(null, b, 0, b.length);
      }
   }

   public static void main(String[] args) {
      if (args.length > 0) {
         var loader = new EasyLoader();
         for (var file : args) {
            System.out.println(file +" ::");
            try {
               var clazz = loader.loadFromDisk(file);
               for (var m : clazz.getMethods()) {
                  if (Modifier.isNative(m.getModifiers())) {
                     System.out.println(m.getName());
                  }
               }
            } catch (IOException | ClassFormatError x) {
               System.out.println("Not a class file");
            }
         }
      }
```

```
    }
  }
```

These types of examples can be fun to explore the dynamic nature of the Java platform and to learn how the Reflection API works. However, it's important that a well-grounded Java developer be conscious of the limitations and occasional frustrations that can occur when working reflectively.

5.3 Problems with reflection

The Reflection API has been part of the Java platform since version 1.1 (1996), and in the 25 years since its arrival, a number of issues and weaknesses have come to light. Some of these inconveniences follow:

- It's a very old API with array types everywhere (it predates the Java Collections).
- Figuring out which method overload to call is painful.
- API has two different methods, `getMethod()` and `getDeclaredMethod()`, to access methods reflectively.
- API provides the `setAccessible()` method, which can be used to ignore access control.
- Exception handling is complex for reflective calls—checked exceptions are elevated to runtime exceptions.
- Boxing and unboxing is necessary to make reflective calls that pass or return primitives.
- Primitive types require placeholder class objects, for example, `int.class`, which is actually of type `Class<Integer>`.
- The `void` methods require the introduction of the `java.lang.Void` type.

As well as the various awkward corners in the API, Java Reflection has always suffered from poor performance for several reasons, including unfriendliness to the JVM's JIT compiler.

NOTE Solving the problem of reflective call performance was one of the major reasons for the addition of the Method Handles API, which we will meet in chapter 17.

There is one final problem with reflection, which is perhaps more of a philosophical problem (or antipattern): developers frequently encounter reflection as one of the first truly advanced techniques that they meet when leveling up in Java. As a result, it can become overused, or a *Golden Hammer* technique—used to implement systems that are excessively flexible or which display an internal mini-framework that is not really needed

(sometimes called the *Inner Framework* antipattern). Such systems are often very configurable but at the expense of encoding the domain model into configuration rather than directly in the domain types.

Reflection is a great technique and one that the well-grounded Java developer should have in their toolbox, but it is not suitable for every situation, and most developers will need to use it only sparingly.

Summary

- The class file format and class loading are central to the operation of the JVM. They're essential for any language that wants to run on the VM.
- The various phases of class loading enable both security and performance features at runtime.
- JVM bytecode is organized into families with related functionality.
- Using `javap` to disassemble class files can help you understand the lower level.
- Reflection is a major feature and extremely powerful.

Previous chapter
< Part 2. Under the hood

Next chapter
5 Java concurrency fundamentals >